Rover — comandos de operação

ROS 2 Jazzy • Gazebo Harmonic • Raspberry Pi 5 • MID360 físico

Revisão: 13/11/2026. Execute no PC em `~/ardupilot/Rover`, salvo indicação “na Raspberry” ou “no MAVProxy”. Cada serviço contínuo ocupa um terminal. Este guia reúne os comandos dos componentes atuais; opções completas e resultados dos ensaios ficam nos tutoriais de cada componente.

Escolha o fluxo

- Simulação local: página 2. Gazebo, SITL e serviços no PC.
- Simulação com controle remoto: página 3. Gazebo/SITL no PC, serviços na Pi.
- Missão integrada e docking: páginas 4 e 5.
- SLAM, câmera, LiDAR e obstáculos simulados: página 6.
- MID360 físico independente: aquisição na página 7; FAST-LIO na página 8.
- Sintonia, Lua e MAVProxy: página 9.
- Instalação: página 10. Perfil do rover real: página 11.
- Diagnóstico e encerramento: página 12.

Ambiente de simulação local — em cada terminal

```
cd ~/ardupilot/Rover
export ROS_DOMAIN_ID=0
export RMW_IMPLEMENTATION=rmw_fastrtps_cpp
export FASTDDS_BUILTIN_TRANSPORTS=UDPv4
unset ROVER_COMPANION_HOST FASTRTPS_DEFAULT_PROFILES_FILE
source ros2/setup.bash
```

Referência de domínios e conexões

- Domínio 0: simulação local/remota; relógio simulado.
- Domínio 20: perfil do rover real; relógio real.
- Domínio 21: aquisição MID360 independente na Pi.
- Domínio 22: teste independente FAST-LIO processado no PC.
- Domínio 23: visualizar no PC o FAST-LIO independente da Pi.
- SSH atual: `rover@192.168.0.167`. MID360 físico: `192.168.1.161`.
- MAVProxy: QGC 14550; docking/manager 14551; MAVROS 14555;

FAST-LIO/Lua 14556; obstáculos 14557; companion remoto 14600.

Os testes físicos do MID360 não comandam o rover. Execute um controlador de missão/docking por vez. Os comandos de missão, armamento e docking abaixo movimentam o veículo quando suas condições forem satisfeitas.

Índice completo: README.md.

2 · Simulação com controle local

Prepare o ambiente da página 1. Abra um terminal para cada etapa abaixo.

1. Gazebo — apenas MID360 e câmera Samsung

```
./ros2/service.sh gazebo \
  world:=gravel_plane N:=1 start_x:=2.5 start_y:=-2 \
  real_time_factor:=1.0 \
  mid360_enabled:=true samsung_s23_ultra_cam_enabled:=true \
  insta360_enabled:=false hokuyo_enabled:=false
```

Para ocultar a janela do Gazebo, acrescente `gui:=false`.

2. SITL, Lua e MAVProxy — sem desvio

```
./sim_1_gzrover.sh --foreground \
  --add-param-file nrover_sem_desvio.param --no-rebuild
```

Aguarde GPS/EKF. O QGroundControl local recebe UDP 14550.

3 a 7. Demais serviços — um comando por terminal

```
./ros2/service.sh fastlio
./ros2/service.sh mapa --wall-timeout 3
./ros2/service.sh mission-manager --vehicle sitl \
  --no-qr --geometry-process --simulate-charge
./ros2/service.sh rviz
./ros2/service.sh camera
```

O RViz usa `camera_init`; o mapa é `/rover_slam_3d/map`. Após o gerenciador estar pronto, use a página 4 para enviar a missão.

Alternativa: abrir os serviços em janelas separadas

```
./iniciar_simulacao_completa.sh --without-avoidance --no-rebuild
```

Não execute junto com as etapas acima. Esse lançador abre Gazebo, SITL, MAVROS, FAST-LIO, mapa, câmera e RViz; não abre o Mission Manager. Opções adicionais: `--glim`, `--headless`. Com desvio, omita `--without-avoidance`. `start_gz_simple.sh` redireciona ao mesmo lançador ROS 2.

Apenas SITL, sem Gazebo

```
python ../Tools/autotest/sim_vehicle.py -v Rover -f rover-skid \
  -L SEParnaiba --sysid 1 -I 0 \
  --add-param-file nrover_sem_desvio.param \
  --out=udp:127.0.0.1:14550 --out=udp:127.0.0.1:14551
```

Esse modo não produz câmera ou LiDAR. Não o execute junto com outro SITL. Guia: simulação local.

3 · Simulação com controle na Raspberry

No PC ficam Gazebo, SITL, RViz e clientes. Na Pi ficam os serviços de bordo. Encerre as cópias locais de FAST-LIO, mapa e gerenciador antes deste fluxo.

Preparação — em cada terminal do PC

```
cd ~/ardupilot/Rover
export ROS_DOMAIN_ID=0
export ROVER_REMOTE=rover@192.168.0.167
source companion/pc_env.bash
```

Terminal 1 — Gazebo no PC

```
./ros2/service.sh gazebo \
  world:=gravel_plane N:=1 start_x:=2.5 start_y:=-2 \
  real_time_factor:=0.5 \
  mid360_enabled:=true samsung_s23_ultra_cam_enabled:=true \
  insta360_enabled:=false hokuyo_enabled:=false
```

O fator 0,5 reduz a carga de rede; não muda a velocidade configurada em m/s.

Terminal 2 — SITL com saída MAVLink remota

```
ROVER_COMPANION_HOST=192.168.0.167 \
  ./sim_1_gzrover.sh --foreground \
  --add-param-file nrover_sem_desvio.param --no-rebuild
```

Terminal 3 — iniciar a Pi e acompanhar

```
./companion/control.sh up
./companion/control.sh logs
```

Ctrl+C em `logs` fecha somente o acompanhamento.

Terminais 4 e 5 — visualização e operação

```
./ros2/service.sh rviz
```

```
./companion/control.sh status
./companion/control.sh mission Missions/new_argo.plan
./companion/control.sh watch
```

A missão é copiada ao container antes da chamada ROS. Aceitação não confirma conclusão; acompanhe o estado. A repetição da saída na Pi tem limitações registradas no relatório de validação.

Cancelar, recuperar ou encerrar

```
./companion/control.sh cancel
./companion/control.sh status
./companion/control.sh reconcile
```

Use `reconcile` após corrigir a causa da falha. Para encerrar, confirme HOLD, parada e desarmamento, execute `./companion/control.sh down` e depois feche RViz/câmera, SITL e Gazebo com Ctrl+C, nessa ordem. Guia: controle remoto.

4 · Mission Manager — missão, docking e recarga

Simulação local: Gazebo, SITL/MAVProxy, FAST-LIO e mapa já ativos. Na execução remota, use a página 3 para enviar o arquivo do PC.

Iniciar o gerenciador e consultar estado

```
./ros2/service.sh mission-manager --vehicle sitl \
  --no-qr --geometry-process --simulate-charge
```

Em outro terminal preparado com `source ros2/setup.bash`:

```
ros2 topic echo /mission_manager/status --once --full-length
ros2 topic echo /mission_manager/status --full-length
```

Enviar arquivo e iniciar o ciclo

```
./ros2/service.sh mission-start "$PWD/Missions/new_argo.plan"
```

Equivalente por serviço, quando o gerenciador lê arquivos deste PC:

```
ros2 service call /mission_manager/start_file \
  rover_mission_interfaces/srv/StartMission \
  "{mission_file: '$PWD/Missions/new_argo.plan'}"
```

Aceita `.plan` com itens simples e `.waypoints`. O último ponto deve ficar na frente de uma estação cadastrada, a menos de 8 m do centro. Se já estiver estacionado, o gerenciador aguarda a carga, sai de ré 3 m e inicia AUTO. Ao terminar a missão, faz docking, desarma e simula recarga.

Missão já carregada — alternativa ao envio por arquivo

```
ros2 service call /mission_manager/bind std_srvs/srv/Trigger '{}'
ros2 topic echo /mission_manager/status --once --full-length
ros2 service call /mission_manager/start std_srvs/srv/Trigger '{}'
```

Aguarde o resultado de `bind` no status antes de `start`.

Outros serviços

```
ros2 service call /mission_manager/dock std_srvs/srv/Trigger '{}'
ros2 service call /mission_manager/cancel std_srvs/srv/Trigger '{}'
ros2 service call /mission_manager/reconcile std_srvs/srv/Trigger '{}'
```

`dock` solicita docking; `cancel` interrompe; `reconcile` reavalia o estado após corrigir a causa. Não cole os três comandos como uma sequência. Confira `state`, `last_error`, `fresh`, `mode`, `armed`, `inside` e `busy`. HOLD é modo 4; GUIDED é 15. A chamada aceita é assíncrona: leia o status.

Cadastro das estações

```
python python/common/docking_stations.py
```

Cadastro: `estacoes_recarga.txt`. Os arquivos de missão de exemplo continuam em `simulacoes/mission_manager/`. Guia: Mission Manager.

5 · Docking independente — QR ou LiDAR

Use estas rotinas separadamente do Mission Manager. Todos os comandos de entrada/saída passam pelo MAVProxy; não alteram a pose diretamente no Gazebo.

QR: conferir câmera e manter monitor de entrada

```
./ros2/service.sh camera
```

Em outro terminal, faça o diagnóstico e, quando ele terminar, abra o monitor:

```
./ros2/service.sh docking-qr --diagnostico
./ros2/service.sh docking-qr
```

Para observar apenas o QR: `./ros2/service.sh observar-qr`. Para limitar a velocidade: `./ros2/service.sh docking-qr --velocidade 0.15`.

Ativar a entrada — no console MAVProxy, não no Bash

```
mode hold
arm throttle
mode guided
```

Espere a parada após HOLD e a confirmação do armamento antes de GUIDED. O monitor exige telemetria recente, GPS 3D, armamento, GUIDED e uma estação a menos de 8 m. Depois verifica a pose, alinha, entra, para e desarma.

Sair usando QR

Encerre primeiro o monitor de entrada. Com rover dentro, parado e em HOLD:

```
./ros2/service.sh sair-qr
```

A rotina independente por QR recua 2 m e gira 180°. Isso difere da saída integrada do Mission Manager, que recua 3 m sem giro programado. A saída visual não foi revalidada em movimento na última revisão.

LiDAR: pré-requisitos — um terminal por comando

```
./ros2/service.sh fastlio
./ros2/service.sh mapa
```

Com dados recentes, execute o diagnóstico antes da entrada:

```
./ros2/service.sh docking-lidar --diagnostico
./ros2/service.sh docking-lidar
```

Para sair, depois de encerrar a entrada, com o rover dentro e parado:

```
./ros2/service.sh sair-lidar
```

Interromper

Ctrl+C no terminal da rotina solicita parada e restauração dos parâmetros. Confirme HOLD e parada no veículo. Alternativa no MAVProxy: `mode hold`. Não inicie a saída enquanto o monitor de entrada estiver aberto.

Guias: QR e serviços, LiDAR, estações.

6 · Percepção e mapas na simulação

Gazebo e sensores ativos. Inicie cada serviço em um terminal preparado. Mantenha o rover parado durante a inicialização da IMU.

FAST-LIO e mapas

```
./ros2/service.sh fastlio
./ros2/service.sh mapa --wall-timeout 3
./ros2/service.sh rviz
```

A opção 2D está disponível com `./ros2/service.sh mapa2d`. Para ensaio sem envio da pose ao Lua: `./ros2/service.sh fastlio --no-mavlink`. Para receber MAVROS quando necessário: `./ros2/service.sh mavros`.

GLIM — odometria e otimização global

```
./ros2/service.sh glim
./ros2/service.sh rviz-glim
```

Use `glim_map` como Fixed Frame do GLIM; FAST-LIO usa `camera_init`. O GLIM é independente do FAST-LIO para mapear. No conjunto completo, FAST-LIO continua fornecendo odometria contínua ao docking. Mapas não são intercambiáveis.

Câmera, QR e leitura do LiDAR

```
./ros2/service.sh camera
./ros2/service.sh observar-qr
./ros2/service.sh lidar --list
./ros2/service.sh lidar
```

Para salvar uma nuvem sem janela:

```
./ros2/service.sh lidar --headless --snapshot /tmp/mid360.npz
```

Habilitar percepção de obstáculos na simulação

Inicie SITL com o perfil abaixo, em vez do perfil sem desvio. Não abra outro SITL.

```
./sim_1_gzrover.sh --foreground \
  --add-param-file nroverobst.param --no-rebuild
```

Em outro terminal: `./ros2/service.sh obstaculos`. `PRX1: No Data` indica ausência de proximidade: confira sensor, ponte e MAVProxy. O módulo é experimental; as limitações de cobertura estão nos relatórios.

Conferir dados

```
ros2 topic hz /fast_lio/odometry
ros2 topic hz /rover_slam_3d/local_cloud
ros2 topic echo /glim/status --once
ros2 topic hz /glim_ros/odom
```

Tópicos RViz: `/rover_slam_3d/map`, `/rover_slam_3d/local_cloud`; GLIM: `/glim_ros/map`. Ctrl+C encerra cada processo; aguarde a gravação do mapa. Guias: SLAM 3D, GLIM.

7 · MID360 físico — aquisição independente

Sensor `47MCNAH0037261`: IP `192.168.1.161`. Raspberry: `rover@192.168.0.167`, com endereço secundário `192.168.1.50/24` na Ethernet. O sensor precisa de alimentação externa. Este fluxo não comanda o rover.

Iniciar e testar — comandos no PC

```
cd ~/ardupilot/Rover
./mid360/control.sh up
./mid360/control.sh status
./mid360/control.sh test
```

`up` prepara o endereço secundário e inicia somente o driver físico na Pi. Para acompanhar seus logs: `./mid360/control.sh logs`.

Visualizar somente a nuvem bruta

Um terminal para a ponte e outro para o RViz:

```
./mid360/ponte_pc.sh
```

```
./mid360/visualizar.sh
```

Verificar a recepção no PC: `./mid360/testar_pc.sh`. Esse fluxo usa domínio 21 e frame `livox_frame`. Mostra a varredura corrente; não executa SLAM nem acumula pontos compensando movimento.

Parar aquisição

Feche ponte e RViz com Ctrl+C. Depois:

```
./mid360/control.sh down
```

Para parar e remover também o endereço secundário criado pelo teste:

```
./mid360/control.sh network-down
```

Se FAST-LIO da página 8 estiver ativo, pare-o antes de desligar o driver.

Diagnóstico de rede — leitura na Raspberry

```
ssh rover@192.168.0.167 'ip -4 addr show dev eth0'
ssh rover@192.168.0.167 \
  'ping -I 192.168.1.50 -c 3 -W 2 192.168.1.161'
```

`network-down` remove somente o endereço secundário registrado pelo teste; não altera a rota padrão. Os serviços físicos não têm reinício automático. Guia: MID360 físico.

8 · MID360 físico — FAST-LIO separado do rover

Escolha onde processar: Raspberry ou PC. Os comandos abaixo são digitados no PC. Não use os dois estimadores de teste ao mesmo tempo.

Opção A: FAST-LIO na Raspberry, RViz no PC

```
cd ~/ardupilot/Rover
./mid360/fast_lio_raspberry/iniciar.sh
```

Inicia driver, adaptador e FAST-LIO na Pi; abre a visualização no PC. Mantenha o sensor parado durante a inicialização.

Somente a Pi, sem abrir janela:

```
./mid360/fast_lio_raspberry/iniciar.sh --sem-rviz
```

Reabrir a visualização, mantendo o estimador ativo:

```
./mid360/fast_lio_raspberry/visualizar.sh
```

Parar todo este teste nas duas máquinas:

```
./mid360/fast_lio_raspberry/parar.sh
```

Preparar/atualizar os arquivos e compilar o adaptador ARM64:

```
./mid360/fast_lio_raspberry/preparar.sh
```

Domínio 21 na Pi; 23 no PC. RViz: `camera_init`, `/lio_pi/cloud_registered` e `/lio_pi/map_pc`. Fechar RViz mantém FAST-LIO na Pi. Mapa visual de até 300 mil voxels no PC; não é fechamento de laços. Mouse: Move Camera + Orbit.

Opção B: driver na Pi, FAST-LIO e RViz no PC

```
./mid360/control.sh up
./mid360/fast_lio_local/build.sh
./mid360/fast_lio_local/iniciar.sh
```

`build.sh` é necessário na preparação/atualização, não a cada execução. Sem RViz: acrescente `--no-rviz` a `iniciar.sh` desta opção.

```
./mid360/fast_lio_local/parar.sh
./mid360/control.sh down
```

Domínio 22 no PC. Nesta opção, fechar o RViz também encerra o teste local. Os scripts carregam o Python compatível com Jazzy automaticamente. Guias: FAST-LIO na Pi, FAST-LIO no PC.

9 · Sintonia, Lua e MAVProxy

As chamadas de sintonia abaixo são alternativas entre si. O endpoint deve ser uma saída exclusiva do MAVProxy; não compartilhe a porta com MAVROS.

Missão circular e quadrado circunscrito

```
./mission_tune.sh --help
./mission_tune.sh --radius 3
./mission_tune.sh --radius 3 --upload-only
./mission_tune.sh --radius 3 --execute --wait-qgc
./mission_tune.sh --radius 3 --execute
./mission_tune.sh --radius 3 --execute --optimize
```

Somente `--radius` gera os arquivos offline. `--execute` envia, arma e executa; `--optimize` testa ajustes de controle. Raio R gera quadrado de lado 2R. O lançador atual usa Python 3.12. Os resultados anteriores à migração ROS 2/Gazebo Harmonic são históricos, não validação da física atual.

Configurar saída exclusiva — no MAVProxy

O autotuning usa 14555 por padrão, mas essa porta atende MAVROS no conjunto atual. Com MAVROS ativo, use uma porta livre, por exemplo 14558:

```
output add udp:127.0.0.1:14558
set mavfwd true
```

No Bash, selecione a mesma porta:

```
./mission_tune.sh --radius 3 --execute \
  --endpoint udpin:127.0.0.1:14558
```

Modos e inspeção — no MAVProxy

```
mode hold
mode guided
mode auto
param show WP_SPEED
param show SCR_ENABLE
wp list
```

São comandos individuais. Não cole mudanças de modo em sequência durante uma manobra. Modos Rover: HOLD 4; AUTO 10; GUIDED 15.

Lua e firmware

O perfil escolhido em `--add-param-file` fornece parâmetros de inicialização; `sim_1_gzrover.sh` prepara os scripts Lua conforme o lançador atual. Código em `scripts/`, documentação em `tutoriais/lua/`.

Para recompilar o firmware com SITL encerrado, a partir de `~/ardupilot`:

```
/home/grin/.pyenv/versions/3.8.10/bin/python3.8 ./waf rover -j4
```

Python 3.8 aqui é do Waf desta árvore; serviços ROS 2 usam Python 3.12. Guias: sintonia, Lua.

10 · Instalação e atualização

Execute somente ao preparar ou atualizar a instalação, com os serviços correspondentes encerrados. Esses comandos não iniciam uma missão.

PC: ROS 2 e componentes

```
cd ~/ardupilot/Rover
sudo bash ros2/install_ros2.sh
./ros2/build.sh
source ros2/setup.bash
```

Ajustar os buffers DDS uma vez; reinicie os processos ROS depois:

```
sudo bash companion/configure_dds_buffers.sh
```

Raspberry nova: enviar instaladores a partir do PC

```
scp companion/install_docker.sh companion/configure_dds_buffers.sh \
  rover@192.168.0.167:
ssh rover@192.168.0.167
```

Na Raspberry com Ubuntu 24.04 ARM64 e instalação nova de Docker:

```
sudo bash ~/install_docker.sh --classic-store
sudo bash ~/configure_dds_buffers.sh
exit
```

`--classic-store` é somente para Docker ainda não instalado. Não troque o armazenamento de uma instalação existente. Reabra SSH para atualizar o grupo.

Enviar fontes e compilar a imagem — no PC

```
cd ~/ardupilot/Rover
export ROVER_REMOTE=rover@192.168.0.167
./companion/deploy.sh 2>&1 | tee /tmp/rover-pi5-build.log
```

O deploy preserva configurações existentes e não inicia missão. Para verificar:

```
ssh rover@192.168.0.167 \
  'docker run --rm rover-companion:jazzy python companion/tests/image_smoke.py'
ssh rover@192.168.0.167 'df -h /; free -h; docker compose version'
```

Preparar o teste físico independente

Depois que a imagem existir na Pi:

```
./mid360/fast_lio_raspberry/preparar.sh
```

Guias: instalação ROS 2, Pi 5 com simulação.

11 · Perfil do rover real

Referência de operação após comissionamento do hardware. Os caminhos de serial, calibração, sensores e carregador precisam corresponder ao veículo real. Não envie a missão `new_argo.plan` de simulação ao veículo físico.

Pré-requisitos

Preencha `config/real.yaml` e `config/hardware.yaml` na Pi, configure serial, extrínsecos, sensores, estação e carregador. Marque `hardware_configured: true` somente após conferir a instalação. O tutorial contém a sequência completa. O teste MID360 das páginas 7–8 é independente e não prepara esse perfil.

Iniciar — na Raspberry

```
cd ~/rover_companion
docker compose down
export ARDUPILOT_DEVICE=/dev/serial/by-id/SEU_AUTOPILOTO
docker compose -f compose.yaml -f compose.real.yaml up -d
bash source/companion/logs.sh
```

Troque `SEU_AUTOPILOTO` pelo dispositivo correto. Use sempre os dois arquivos Compose para este perfil. `companion/control.sh up` seleciona simulação.

Status e visualização — no PC

```
cd ~/ardupilot/Rover
export ROVER_REMOTE=rover@192.168.0.167
export ROS_DOMAIN_ID=20
source companion/pc_env.bash
./companion/control.sh status
rviz2 -d ros2/config/mid360_3d.rviz --ros-args -p use_sim_time:=false
```

Missão do local real — em outro terminal preparado

```
./companion/control.sh mission /caminho/da/missao_real.plan
./companion/control.sh watch
```

A interface é a mesma; calibração, sensores, mapa/estação e recarga são do hardware. Não use `--simulate-charge` para representar carga física.

Encerrar

No PC, ainda no domínio 20:

```
./companion/control.sh cancel
./companion/control.sh status
```

Confirme HOLD, parada e desarmamento. Depois, na Raspberry:

```
cd ~/rover_companion
docker compose -f compose.yaml -f compose.real.yaml down
```

Feche RViz e clientes. Para retornar à simulação, abra os terminais no domínio 0. Guia: instalação e comissionamento real.

12 · Diagnóstico e parada

Comandos ROS exigem o ambiente e o domínio do fluxo em uso.

Inspecionar serviços e tópicos

```
ros2 node list
ros2 topic list
ros2 service list
ros2 topic echo /mission_manager/status --once --full-length
ros2 topic hz /rover_argo_N1/Instance1/livox/imu
ros2 topic hz /fast_lio/odometry
ss -ltnp 'sport = :5760'
```

Se 5760 já estiver ocupada, confira o SITL existente antes de abrir outro. Se houver `FALHA`, leia `last_error`, corrija a causa e solicite `reconcile`.

HEARTBEAT ausente na Raspberry

Inicie SITL com `ROVER_COMPANION_HOST=192.168.0.167`. Para acrescentar a saída a um MAVProxy já aberto, sem outra ponte ativa, digite nele:

```
output add udp:192.168.0.167:14600
set mavfwd true
```

Depois, no Bash: `./companion/control.sh reconcile` e `./companion/control.sh status`. A cópia do arquivo por SSH não testa MAVLink.

Parada da simulação local

```
ros2 service call /mission_manager/cancel std_srvs/srv/Trigger '{}'
ros2 topic echo /mission_manager/status --once --full-length
```

Confirme HOLD, parada e desarmamento. Use Ctrl+C nos terminais, nesta ordem: Mission Manager/docking → mapa → FAST-LIO/GLIM → câmera/RViz → SITL → Gazebo. Aguarde a exportação dos mapas. Ctrl+Z apenas suspende, não encerra.

Parada do controle remoto

```
./companion/control.sh cancel
./companion/control.sh status
```

Após confirmar parada/desarmamento:

```
./companion/control.sh down
```

Depois feche visualizadores, SITL e Gazebo locais. Ctrl+C em `logs` ou `watch` encerra só o acompanhamento, não os serviços na Pi.

Parada do teste físico MID360 + FAST-LIO na Pi

```
./mid360/fast_lio_raspberry/parar.sh
```

Esse comando encerra apenas o teste físico independente. Para conferir:

```
ssh rover@192.168.0.167 \
  'docker ps --filter name=rover-mid360 --format "{{.Names}}: {{.Status}}"'
```

Ajuda dos lançadores: `./ros2/service.sh help`, `./iniciar_simulacao_completa.sh --help`, `./mission_tune.sh --help`. Índice de todos os tutoriais e relatórios: README.md.